LinkKeeper — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and the three building blocks

An MCP server plus a client that uses it:

- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py` — settings, named errors (`EmptyBookmarkError`), one logger that writes to **stderr**.
- `src/db.py` — the SQLite store: `init_db`, `add_bookmark` (raises on empty url), `search_bookmarks`, `all_bookmarks`.
- `server.py` — one `MCPServer` with the three building blocks and `mcp.run(transport="stdio")`.
- `client.py` — connects, discovers, uses all three.

The three building blocks (Doc06): **Tools** (`@mcp.tool()` — actions), a **Resource** (`@mcp.resource("bookmarks://all")` — readable content by URI), and a **Prompt** (`@mcp.prompt()` — reusable wording).

Build order: a minimal one-tool server → the SQLite store + search → add the resource + prompt → the client.

Hint 2 — the server, and the stdout rule that bites everyone

```
from mcp.server.mcpserver import MCPServer
mcp = MCPServer("linkkeeper-server")

@mcp.tool()
def add_bookmark(url: str, title: str, tag: str = "general") -> str:
    """Save a bookmark..."""        # the docstring IS the tool description
    ...

@mcp.resource("bookmarks://all")
def all_bookmarks_resource() -> str: ...

@mcp.prompt()
```

```
def reading_list() -> str: ...

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

The rule: never `print()` to stdout in the server — stdout is the MCP protocol channel, and a stray print corrupts the client's parsing. Log to **stderr** (`logging.StreamHandler(sys.stderr)`), and if you must print a startup line, `print(..., file=sys.stderr)`.

Hint 3 — returning an error the client can read

In the current `mcp` SDK, a tool that **raises** gives the client only a generic "tool failed." So catch the error and **return a string**:

```
try:
    db_add(CONFIG.db_path, url, title, tag)   # raises EmptyBookmarkError
except EmptyBookmarkError as e:
    return f"Error: {e}"
return f"Saved: {title} [{tag}]"
```

The client then checks the text: `result.content[0].text.startswith("Error")` — version-independent, because the message is a plain string you control. (The SDK's own flag is `result.is_error`, snake_case — but the string check is simpler and always works.)

Hint 4 — the client: discover, then use; and persistence

The client never imports from the server — it **discovers** over the protocol:

```
async with stdio_client(params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()                      # handshake first
        (await session.list_tools()).tools              # discover
        (await session.list_resources()).resources      # -> r.uri
        (await session.list_prompts()).prompts           # -> p.name
        added = await session.call_tool("add_bookmark", {...})
        added.content[0].text                            # use a tool
        res = await session.read_resource("bookmarks://all")
        res.contents[0].text                             # read the resource
        pr = await session.get_prompt("reading_list")
        pr.messages[0].content.text                      # get the prompt
```

Note the shapes: a tool result has `.content[0].text`; a **resource** read has `.contents[0].text` (note the plural `contents`); a prompt has `.messages[0].content.text`. And because the store is SQLite on disk, add a bookmark, stop the server, start it again, and search — it's still there. The `test_db` "persists on disk" case and a manual restart both prove it.

Still stuck? The whole server and client, every file, is in the complete solution.